

Rapport de Projet : Modèle Génératif Local pour l'Exploration et la Chasse Collaborative

Lionel ALVES VIEIRA & Thomas LU

18 mai 2026

Table des matières

1	Introduction	2
2	État de l'Art et Choix Technologiques	3
2.1	L'évolution des architectures d'agents basées sur les LLM	3
2.2	Paradigmes de communication et coordination multi-agents	4
2.3	Sélection et contraintes des modèles locaux (Small Language Models)	4
3	Architecture : La Boucle Perception-Décision-Action	5
3.1	La Perception : Structuration du contexte et asynchronisme	5
3.2	Le Cœur Cognitif : Cadrage du LLM par LangChain4j	5
3.3	L'Action et la Sécurisation : Les Garde-fous Symboliques	6
4	Progression Expérimentale et Cas d'Usage	7
4.1	Cas Mono-Agent : Exploration pure et gestion de la latence	7
4.2	Cas Multi-Agents Décentralisé : Coopération et transition organique	7
5	Évaluation et Benchmarking	8
5.1	Analyse de la Latence : Slow-Start et vitesse de croisière	8
5.2	Impact de la charge Multi-Agents (Scalabilité)	9
5.3	Évaluation de la phase de Chasse : La surcharge cognitive	10
5.4	Le Crash-Test à l'Extrême : Simulation à 7 Agents simultanés	11
6	Discussion, Limites et Perspectives	12
6.1	Limites Cognitives et Logicielles	12
6.2	Perspectives d'Amélioration	13
7	Conclusion	13

1 Introduction

L'intelligence artificielle a connu des avancées spectaculaires avec l'émergence des Grands Modèles de Langage (*Large Language Models* - LLM), ouvrant de nouvelles perspectives quant à leur intégration au sein de systèmes autonomes. Historiquement restreints à des tâches de traitement du langage naturel, ces modèles génératifs sont désormais envisagés comme des moteurs de décision embarqués dans une boucle "*perception-décision-action*". C'est dans ce contexte que s'inscrit notre projet, dont l'objectif est d'explorer l'utilisation d'un modèle génératif local pour le contrôle stratégique d'agents autonomes. Les tâches retenues pour évaluer cette architecture sont l'exploration d'un environnement inconnu et la chasse d'un adversaire sur la plateforme Dédale [1].

La plateforme Dédale est un environnement de simulation pour les Systèmes Multi-Agents (SMA), matérialisé sous la forme d'un graphe topologique inter-connecté. Dans cet univers contraint, les agents physiques ne disposent que d'une visibilité locale et immédiate (son nœud courant et ses voisins). Pour interagir avec leur milieu, ils doivent s'appuyer sur des primitives logiques natives et asynchrones, principalement la méthode `observe()`, qui renvoie la liste des nœuds adjacents ainsi que les observations associées - présence de ressources (ne nous intéresse pas ici) ou d'entités (les adversaires) -, et `moveTo()`, permettant d'effectuer un déplacement d'un nœud à un autre nœud adjacent. Le défi est à son apogée lors des phases de chasse, où les agents doivent localiser et encercler de manière coordonnée un adversaire dynamique et fuyant, le Golem, dont le comportement et la position est par nature imprévisible et inconnue par les agents.

La problématique centrale de notre étude consiste à déterminer comment doter les agents de capacités de raisonnement dans leur environnement et se coordonner à l'aide de modèles minimalistes. Initialement, notre approche reposait sur du *prompting textuel pur* : le LLM agissait de manière totalement disjointe de l'environnement, générant des réponses en langage naturel que notre code Java analysait, nettoyait et parsait manuellement afin d'en déduire l'action à exécuter. Par exemple, le LLM renvoyait l'ID d'un nœud pour indiquer sur quel nœud faire le déplacement. Pour pallier ses limites, et parce que les études montrent que cette méthode n'est pas la plus efficace, notre architecture finale place le modèle directement au cœur cognitif du système via le mécanisme de *Function Calling*. Contrairement aux approches où le LLM génère directement du code exécutable complet, le modèle traduit ici directement ses choix stratégiques en appels directs et sécurisés des fonctions natives de l'API Dédale.

La réalisation de ces objectifs se heurte à des contraintes techniques particulièrement fortes. La principale exigence de ce projet est l'exécution locale, ce qui garantit l'indépendance de notre architecture vis-à-vis des API externes et minimise la latence liée à l'inférence. Cette obligation impose d'utiliser un modèle suffisamment léger — tel que Llama 3.2 3B, BitNet ou Mistral pour n'en citer que trois — car le coût en calcul (RAM et VRAM) peut rapidement devenir problématique lors de la multiplication du nombre d'agent simulés, surtout lorsque les spécifications matérielles de la machine exécutant le programme ne sont pas connues à l'avance.

Ce rapport détaille notre progression architecturale, évoluant d'un concept d'exploration mono-agent jusqu'aux phases de chasse collaborative en environnement multi-agents, en mesurant l'efficacité de l'IA face aux contraintes du monde simulé.

2 État de l'Art et Choix Technologiques

Les LLM, et plus particulièrement l'utilisation de modèles de langage comme contrôleurs d'agents autonomes au sein d'environnements virtuels, représente un axe de recherche en pleine expansion. Cette section dresse un panorama des connaissances actuelles et de l'état de l'art concernant l'interfaçage entre les agents cognitifs et les modèles génératifs, les paradigmes de communication multi-agents, ainsi que les critères de sélection des modèles locaux.

L'étude approfondie de Plaat et al. (2025) dans leur article "*Agentic Large Language Models, a survey*" [2] constitue aujourd'hui une taxonomie très détaillée de ce domaine de recherche. Ils y définissent formellement les LLMs dits "agentiques" (*Agentic LLMs*) non plus comme de simples générateurs de texte passifs, mais comme des entités autonomes capables d'orchestrer un cycle vertueux autour de trois piliers fondamentaux : **raisonner** (*reason*), **agir** (*act*), et **interagir** (*interact*). C'est ce triptyque conceptuel, transcendant la simple complétion de prompt, qui guide notre approche pour intégrer ces modèles au sein de la plateforme Dédale.

2.1 L'évolution des architectures d'agents basées sur les LLM

Traditionnellement, les agents au sein des Systèmes Multi-Agents (SMA), s'appuient sur des architectures symboliques ou des systèmes à base de règles, comme l'architecture BDI : *Belief-Desire-Intention*) pour planifier leurs actions. L'avènement des modèles génératifs a profondément bouleversé ce paradigme en introduisant des "*agents autonomes génératifs*" [3]. Comme le souligne Morge (2025) et la taxonomie de Plaat (2025), ces agents n'utilisent plus une base de règles figée, mais exploitent la mémoire sémantique et les capacités de raisonnement émergentes d'un LLM pour interpréter des situations complexes.

Dans la littérature scientifique récente, et notamment pour concrétiser la capacité d'action (*act*) de ces modèles, deux grandes approches se confrontent pour l'interfaçage d'un LLM avec son environnement :

1. **Le Prompting textuel direct (Génération de texte libre)** : L'environnement est entièrement traduit sous forme de description textuelle (prompt). Le modèle répond par du texte naturel, nécessitant une couche logicielle intermédiaire de filtrage et d'analyse syntaxique (*parsing*) pour extraire l'action voulue. Bien que flexible, cette méthode souffre d'un manque de robustesse, car le modèle peut dévier du format attendu ou halluciner des actions syntaxiquement incorrectes.
2. **L'appel d'outils de programme (*Function Calling* ou *Tool Use*)** : Le modèle reçoit, en plus du prompt, les signatures des fonctions qu'il peut exécuter. Au lieu de générer une phrase, le modèle produit une structure de données décrivant précisément quelle fonction appeler et avec quels arguments. Cette approche, au cœur de protocoles modernes comme le *Model Context Protocol* (MCP), réduit drastiquement les erreurs de formatage et ancre le modèle dans les contraintes physiques du système.

Des expérimentation récentes menées sur des moteurs de jeu, telles que l'implémentation d'IA tactiques commandant des unités sur le moteur Unity [4], démontrent que le *Function Calling* offre une stabilité indispensable pour l'exécution d'actions spatiales en temps réel, évitant que l'agent ne tente d'exécuter des requêtes impossibles et/ou invalides.

2.2 Paradigmes de communication et coordination multi-agents

Lorsque la tâche requiert la collaboration de plusieurs entités - comme la recherche ou l'encerclement coordonné d'un adversaire -, la dimension d'interaction (*interact*) définie par Plaat devient centrale. Tran et al. (2025)[5] proposent une taxonomie exhaustive et très complète des mécanismes de collaboration basés sur les LLM. Citons-en deux :

- **L'architecture centralisée (ou à agent Leader)** : Un agent centralisé, doté d'une vue d'ensemble ou centralisant les messages de ses pairs, interroge le LLM pour planifier et distribuer les rôles aux agents subordonnés (*Followers*). Cela simplifie grandement le traitement pour le LLM qui n'a qu'un seul flux logique à gérer, mais crée un goulot d'étranglement et un point unique de défaillance (*Single Point of Failure*).
- **L'architecture décentralisée pair-à-pair (*Peer-to-Peer*)** : Chaque agent possède son propre flux cognitif (sa propre instance de LLM ou son propre contexte d'interrogation) et communique avec ses voisins par échange de messages (comme le partage de graphes topologiques via **SHARE-TOPO** dans Dédale). L'intelligence et la coordination émergent alors des interactions locales et de la fusion des connaissances acquises.

2.3 Sélection et contraintes des modèles locaux (Small Language Models)

Le cahier des charges de notre projet impose une contrainte stricte : l'exécution des modèles doit être intégralement locale. Cette restriction élimine d'emblée l'usage d'API cloud propriétaires (comme GPT-4 ou Gemini Ultra) et nous oriente vers le catalogue de modèles ouverts disponibles notamment sur Hugging Face [6], déployables via des serveurs d'inférence légers comme Ollama.

L'évaluation des modèles doit concilier deux facteurs principaux : la performance cognitive (capacité à comprendre un graphe et à respecter un format d'API) et l'empreinte matérielle (utilisation de la RAM et de la VRAM). Dans le cadre d'une simulation multi-agents où chaque entité est susceptible d'instancier son propre modèle, le coût computationnel grandit de façon linéaire.

Quatre candidats majeurs ont été distingués par nous mêmes pour ce type d'intégration *edge computing* :

- **Llama 3.2 3B** : Modèles de Meta optimisés pour les architectures légères. Ils disposent d'une excellente réactivité et d'une gestion native du *Function Calling*, mais leur fenêtre de contexte réduite peut saturer lorsque la taille du graphe topologique augmente.
- **Mistral (7B et variantes Small)** : Reconnu pour ses performances de haut niveau en logique et en mathématiques. Il commet globalement moins d'hallucinations spatiales, mais requiert une configuration matérielle plus robuste (au moins 8 Go de VRAM dédiés), ce qui limite le nombre d'instances simultanées en local.
- **BitNet (Modèles quantifiés à 1.58-bit)** : Une innovation très récente dans le domaine du calcul bas niveau. En réduisant les poids du modèle à des valeurs ternaires (-1, 0, 1), BitNet offre une réduction drastique de l'empreinte mémoire et une accélération de l'inférence sans perte majeure de capacités linguistiques.

- **Gemini Nano / Gemma** : Les modèles légers de Google, optimisés pour s'exécuter directement sur des appareils mobiles ou des configurations matérielles restreintes, offrant un bon compromis entre vitesse d'inférence et structuration des réponses.

3 Architecture : La Boucle Perception-Décision-Action

L'intégration d'un LLM local au sein d'un agent Dédale nécessite une séparation stricte entre le moteur physique de la simulation (qui gère les déplacements et le réseau) et le moteur cognitif (l'inférence du LLM). Pour orchestrer cette séparation, nous avons implémenté une architecture modulaire en trois phases : la structuration de la perception, l'inférence décisionnelle via le framework LangChain4j, et l'exécution sécurisée de l'action.

3.1 La Perception : Structuration du contexte et asynchronisme

Dans la plateforme Dédale, l'agent ne dispose d'aucune vision globale. À chaque itération de son `TickerBehaviour` (`LlmTestBehaviour`), le programme Java effectue un appel à la fonction `observe()` pour collecter les données brutes de son nœud actuel et de ses voisins directs. Afin d'optimiser la fenêtre de contexte restreinte des SLM (*Small Language Models* — nos modèles locaux d'environ 3 milliards de paramètres étant trop légers pour l'appellation LLM classique)- et de limiter la latence de traitement, ces données sont "pré-mâchées" par notre architecture :

- **Filtre géométrique et Tri topologique** : Lors de nos premiers tests, nous avons constaté que le modèle souffrait d'un conflit entre la *macro-stratégie* (compréhension du graphe global) et la *micro-action* (faire un pas physique valide). Face à un contexte trop large, l'agent était victime d'hallucination et voulait se "téléporter". Pour pallier ce manque de logique géométrique, la perception liste strictement les `allNeighbors` (les voisins physiques immédiats). Ces derniers sont ensuite divisés entre `newNeighbors` (jamais visités ; prioritaires pour l'exploration) et `oldNeighbors` (déjà visités ; pour le repli).
- **Analyse sensorielle** : La présence d'odeurs (*Stench*) émises par le Golem est extraite et n'est pas envoyée sous forme de données brutes, mais traduite en aver-tissements textuels clairs et quantifiés : (ex : "*Node 35 has a suspicious smell*" ou "*Node 12 smells VERY STRONG (3 odors)*"). Cette traduction sémantique facilite grandement l'interprétation pour le SLM.
- **Le système Radio (Inbox asynchrone)** : La communication multi-agents est gérée de manière totalement désynchronisée du LLM. Un comportement JADE dédié, le `ListenerBehaviour`, écoute en permanence le réseau. Il intercepte les messages textuels tactiques (LLM-CHAT) qu'il stocke dans la "boîte de réception" (*Inbox*) de l'agent, et fusionne silencieusement en arrière-plan les graphes reçus (*SHARE-MAP*) via la méthode `mergeMap()`. Ainsi, lorsque l'IA est sollicitée, elle dispose instantanément d'une vision unifiée et à jour de la topologie partagée.

3.2 Le Cœur Cognitif : Cadrage du LLM par LangChain4j

La prise de décision est déléguée au modèle *Llama 3.2 3B*, hébergé localement via le serveur Ollama. Le pont entre le code Java et l'API du modèle est assuré par la bibliothèque `LangChain4j`.

L'interface **AgentBrain** joue le rôle de chef d'orchestre cognitif. Elle est annotée par un **@SystemMessage** qui conditionne le comportement du LLM. L'ingénierie de ce prompt s'est révélée critique. Nos premières itérations incluait des interdictions strictes (le *prompting négatif*, ex : "Ne fais pas de roleplay", "N'utilise pas le mot K.O."). Or, cette approche sature l'attention d'un SLM à cause du mécanisme d'attention des tokens : en lisant une interdiction, le modèle active les tokens, que l'on voulait interdire, dans son réseau de neurones, augmentant paradoxalement les chances de les générer. L'architecture finale privilégie un lexique purement mathématique, logique et directif ("*Containment and exploration mission*", "*Mathematical encirclement*", etc.), encadrant l'IA dans un rôle algorithmique strict. De plus, le prompt est écrit en anglais, puisque les LLM sont entraînés pour répondre à des prompts dans cette langue.

La communication avec l'environnement physique repose entièrement sur le paradigme du *Function Calling*. Les actions disponibles sont définies dans la classe **DedaleTools** et exposées au LLM via l'annotation **@Tool**. L'IA est ainsi capable de choisir dynamiquement entre :

1. **executeMove(nodeId)** : pour se déplacer.
2. **sendMessage(content)** : pour avertir ses alliés d'un danger.
3. **pingNearbyAgents()** : pour déclencher un partage topologique.

Cependant, le *Function Calling* a mis en exergue une composante importante du Prompt-Engineering : le petit modèle Llama 3.2 3B peine parfois à respecter des schémas JSON complexes imposés par l'outil. Il lui est arrivé de confondre la définition du schéma d'entrée avec la valeur à retourner (ex : renvoyer l'objet `{description=16, type=string}` au lieu de la chaîne "16"). En modifiant le prompt d'initialisation de l'agent pour être plus précis, ce problème s'est réglé, mais cela montre encore l'importance de la bonne rédaction des prompts donnés au LLM.

3.3 L'Action et la Sécurisation : Les Garde-fous Symboliques

L'un des défis majeurs posés par l'usage d'un modèle génératif est son caractère probabiliste. Une IA peut paniquer, mal formater sa réponse JSON, ou halluciner des nœuds inexistants. Pour pallier cette instabilité, la boucle d'action intègre un "bouclier" symbolique au niveau du code Java.

- **Le mécanisme de Fallback (Mouvement de secours)** : Dans nos toutes premières versions, le parsing de la réponse textuelle du LLM avec un simple `.contains()` causait des faux positifs mortels (l'agent se déplaçait en "2" quand l'IA écrivait "26"). Aujourd'hui, la réponse est rigoureusement validée contre la liste `allNeighbors`. Si l'IA échoue (ID inexistant ou erreur JSON), un bloc `catch` intercepte la réponse. Au lieu de laisser l'agent paralysé (créant un *Single Point of Failure* pour l'équipe), le code Java prend le relais et force un "*mouvement d'urgence*" (**Emergency fallback**) vers le premier nœud (dans `newNeighbor` en priorité) disponible. Ce garde-fou certifie la continuité de l'exploration. L'IA guide la stratégie, mais le code Java garantit le respect de la physique du jeu.
- **La mécanique de collision lors de la chasse** : Durant la phase de chasse, le LLM identifie parfois le Golem sur un nœud adjacent (ex : nœud 24) et utilise l'outil `executeMove(24)` dans l'intention de l'attraper. Bien que la manœuvre semble syntaxiquement valide, le moteur physique de Dédale interdit à deux entités d'occuper la même case. La requête de déplacement renvoie alors une erreur (**Move failed**).

Néanmoins, ce comportement devient une force tactique : l'agent reste immobile sur la case adjacente au Golem, ce qui constitue physiquement un barrage (blocage) parfait contre l'adversaire, servant ainsi naturellement les objectifs d'encerclement sans nécessiter l'ajout d'une fonction d'attente explicite pour ce petit modèle.

4 Progression Expérimentale et Cas d'Usage

Pour évaluer la pertinence et la robustesse d'une architecture orientée agents autonomes génératifs (*Agentic LLMs*), nous avons structuré nos expérimentations selon une complexité croissante, alignée sur les jalons du cahier des charges. Cette approche permet d'isoler l'impact du traitement du langage dans différents contextes de coopération.

4.1 Cas Mono-Agent : Exploration pure et gestion de la latence

Le premier cas d'usage se concentre sur un agent unique naviguant seul dans l'environnement inconnu, sans cible dynamique quelconque. L'objectif était de valider la viabilité de la boucle Perception-Décision-Action à l'échelle d'un seul *Small Language Model* (SLM), tout en observant son comportement lorsque son but est d'explorer l'entièreté d'un environnement inconnu.

Les résultats de ce cas d'école ont mis en lumière deux phénomènes critiques au niveau des temps d'inférence que l'on détaillera dans la partie "Évaluation et Benchmarking".

Sur le plan comportemental, l'agent parvient à cartographier son environnement de manière organique, guidé par la distinction sémantique entre `newNeighbors` et `oldNeighbors`. Cependant, un SLM est un réseau de neurones, privé de structures de données comme une pile, l'empêchant par la même occasion d'exécuter des algorithmes de recherche rigoureux (comme DFS ou A*). Lors de nos premiers tests, face à un cul-de-sac, l'agent oubliait les embranchements laissés plusieurs tours en arrière et avait tendance à retourner en arrière naïvement.

Pour pallier cela, nous avons intégré, sans même le vouloir, un mécanisme de ***Chain-of-Thought (CoT)***. En modifiant l'outil de déplacement de l'API Java pour exiger un paramètre `explication` (une phrase d'explication obligatoire) - qui devait au départ juste servir pour étudier son comportement à l'exploration -, nous avons forcé le LLM à verbaliser sa logique avant de fournir l'ID du nœud de destination. Cette étape intermédiaire a corrigé l'oscillation et a rétabli une exploration exhaustive purement générative.

4.2 Cas Multi-Agents Décentralisé : Coopération et transition organique

Notre implémentation s'est ensuite dirigée vers le déploiement de plusieurs agents, chacun piloté par sa propre instance locale du SLM, évoluant face à un adversaire mobile. Au départ, nous avons déployé de deux agents (`OllamaBot1` et `OllamaBot2`), évoluant face à un adversaire mobile (`Wumpus1`).

D'après nos traces d'exécution, la simulation se déroule selon trois phases comportementales émergentes :

1. **Cartographie asynchrone** : Au départ, les agents explorent des zones distinctes. Grâce au comportement `ListenerBehaviour`, les cartes se fusionnent silencieusement en tâche de fond. L'Inbox textuelle de l'agent reçoit des notifications cli-

niques (Map updated by: OllamaBot2), permettant aux deux entités d'étendre leur connaissance du monde sans saturer leur bande passante cognitive.

2. **Analyse sensorielle et convergence** : Dès que l'un des agents capte une odeur suspecte (*Stench Analysis*), le ton de la communication change. Les messages radio se synchronisent : "*Warning : Golem detected on node 35. Occupying node 36, preparing to block.*"
3. **Encerclement émergent et limites physiques** : Lorsque le Golem est formellement repéré (ex : sur le nœud 24), l'agent OllamaBot1 tente un déplacement direct sur sa position (`executeMove(24)`). Le moteur de Dédale rejette légitimement l'action (`Move failed`), empêchant la superposition physique. De façon tout à fait émergente, cet échec fige l'agent sur la case adjacente, créant un barrage.

Malgré ce succès tactique, ce cas d'usage a révélé une faiblesse : n'ayant pas de coordination spatiale croisée en temps réel dans leur "pensée", les deux SLM ont parfois provoqué des embouteillages mutuels sur des nœuds voisins.

5 Évaluation et Benchmarking

Afin de quantifier la viabilité de notre architecture basée sur des *Small Language Models* (SLM) en environnement multi-agents, nous avons mis en place un protocole de log automatisé générant des fichiers CSV. À chaque itération de la boucle de décision, l'agent enregistre le temps d'inférence (latence), la taille du graphe connu et le type d'actions choisies.

Pour garantir la cohérence et la comparabilité de toutes nos mesures, **l'intégralité de nos expérimentations a été réalisée exclusivement avec le modèle Llama 3.2 (3B paramètres)**. Toutes les simulations ont été exécutées localement sur une machine de test unique dotée de la configuration matérielle suivante :

- **Processeur (CPU)** : Intel Core Ultra 7 155H ;
- **Carte Graphique (GPU)** : NVIDIA GeForce RTX 4070 Laptop GPU ;
- **Mémoire Vive (RAM)** : 16 Go de mémoire vive couplée à un stockage système NVMe.

5.1 Analyse de la Latence : Slow-Start et vitesse de croisière

L'un des défis majeurs de l'IA embarquée et/ou locale est le temps de réponse. Nos métriques démontrent un comportement très marqué (Figure 1) :

- **Démarrage à froid** : Le premier tour d'inférence exige un temps de traitement plutôt long, mesuré en moyenne autour de **9000 ms** (avec des pics à 9312 ms). Ce pic s'explique par le chargement matériel des poids du modèle (environ 3 Go) dans la VRAM/RAM de la machine, combiné à l'analyse initiale (*Prompt Processing*) du long `SystemMessage` définissant les règles du jeu.
- **Vitesse de croisière** : Dès le second tour, la latence s'effondre de manière spectaculaire pour atteindre une moyenne de **3500 ms** (mesure effectuée sur le benchmark mono-agent). Le serveur Ollama utilise un mécanisme de KV Cache (*Key-Value Cache*) qui conserve en mémoire les activations des tokens statiques (les instructions globales). Ainsi, le SLM n'a besoin de traiter activement que le nouveau contexte

(les nouveaux voisins observés), rendant le système compatible avec les contraintes du temps réel.

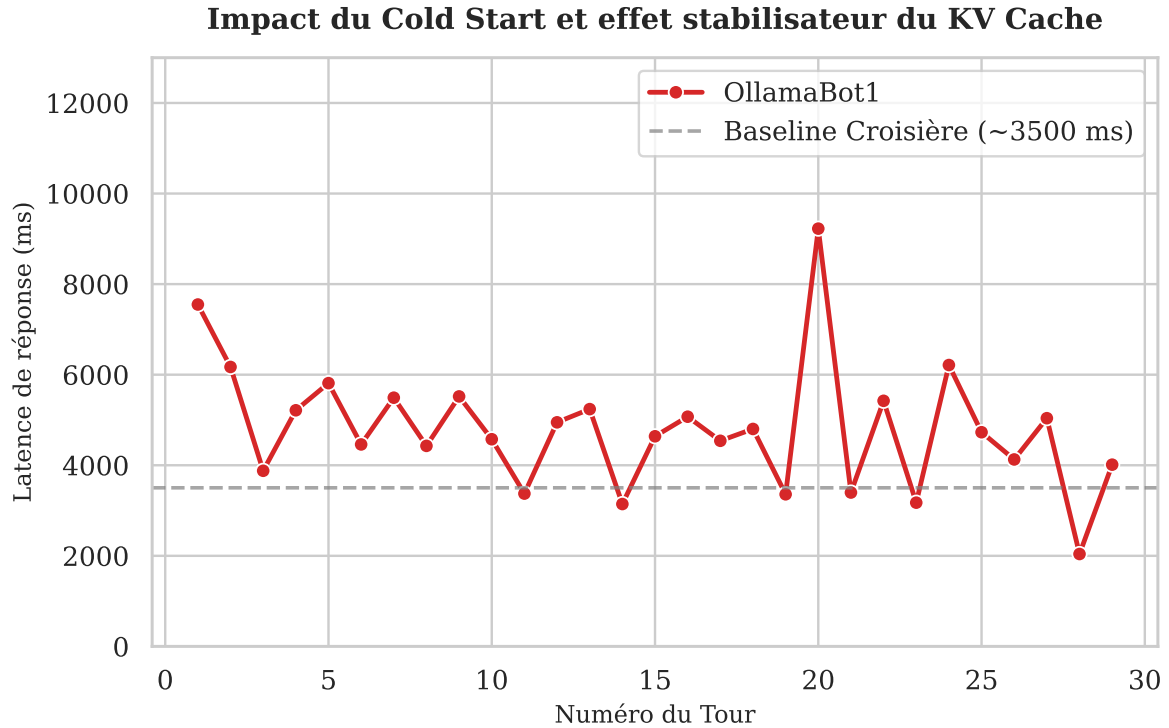


FIGURE 1 – Impact du Cold Start et effet stabilisateur du KV Cache au fil des tours (Modèle Llama 3.2 3B).

Pour valider la robustesse de cette vitesse de croisière, nous avons mesuré la latence sur une simulation à long terme s'étendant sur plus de 75 tours (Figure 2). L'absence complète de dérive ou d'augmentation du temps de traitement confirme que le KV Cache maintient une efficacité constante et un temps de traitement prédictible, tant que la taille limite de la fenêtre de contexte local n'est pas franchie.

5.2 Impact de la charge Multi-Agents (Scalabilité)

Dans notre architecture décentralisée, chaque agent possède son propre "cerveau", c'est à dire sa propre instance LangChain4j, mais tous requêtent le même serveur local Ollama en arrière-plan. Nous avons comparé la latence moyenne en phase d'exploration pure entre un déploiement à 1 agent et un déploiement à 2 agents.

Les résultats (Figure 3) montrent que le temps de réponse moyen passe de **3502 ms** (mono-agent) à **3794 ms** (multi-agents). Cette augmentation mesurée à environ 300 ms est une excellente nouvelle. Elle indique que les **TickerBehaviour** des agents sont naturellement désynchronisés : ils n'interrogent pas l'API d'Ollama exactement à la même milliseconde. Le serveur local parvient donc à traiter les requêtes sans créer de goulot d'étranglement majeur sur la mémoire, ce qui valide la scalabilité de l'approche décentralisée pour des petites escouades.

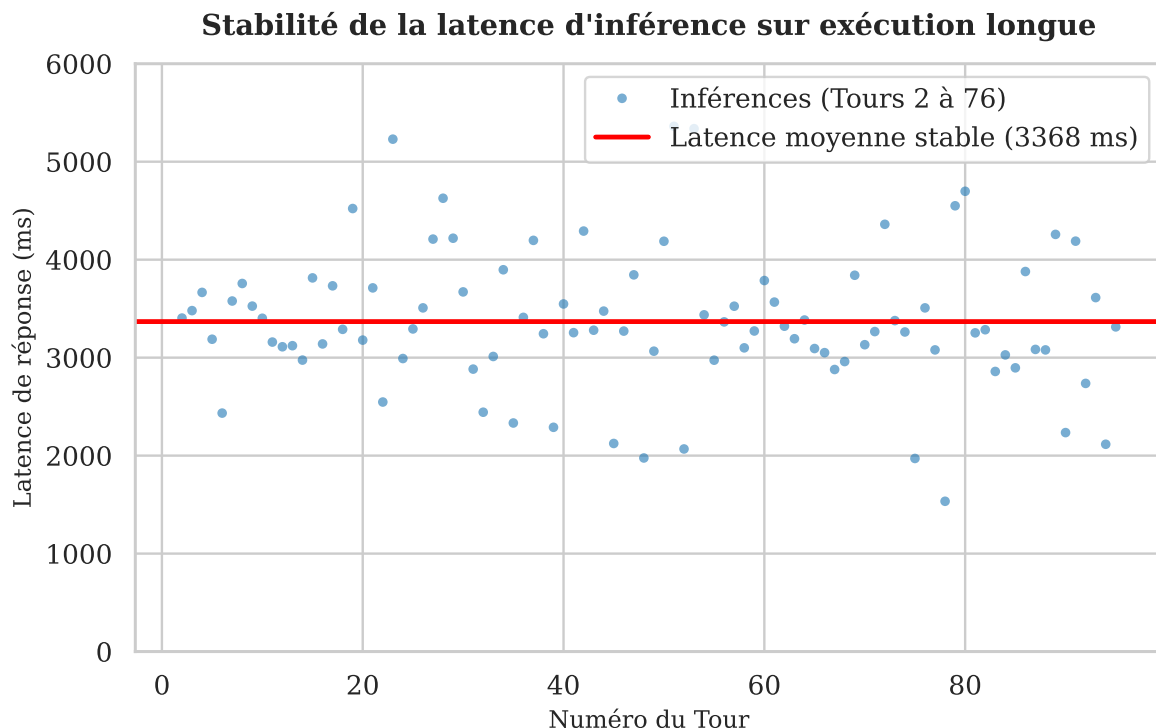


FIGURE 2 – Stabilité de la latence d'inférence sur une exécution longue (75+ tours).

5.3 Évaluation de la phase de Chasse : La surcharge cognitive

La phase de chasse au Golem introduit un poids supplémentaire sur le modèle. Lorsqu'un agent détecte une odeur (*Stench*) ou croise visuellement la cible, la taille de son prompt / de sa fenêtre de contexte augmente drastiquement. L'agent doit traiter les alertes système (!!! TARGET IN SIGHT !!!), analyser l'historique de la radio, et justifier son choix tactique via le mécanisme de *Chain-of-Thought* maintenant imposé dans l'outil de déplacement.

L'analyse de la colonne conditionnelle **IsHunting** dans nos logs démontre que cette "surcharge cognitive" impacte directement la latence. Le temps de décision grimpe en moyenne à **4635 ms** durant la manœuvre d'encerclement (voir la troisième colonne de la Figure 3, +840 ms par rapport à l'exploration pure). Néanmoins, cet investissement temporel est compensé par une fiabilité accrue : le forçage de la verbalisation (l'outil exigeant le paramètre de justification **explication**) oblige le modèle à aligner sa logique spatiale avant d'agir, évitant ainsi les oscillations probabilistes ("essuie-glaces") et facilitant la création de barrages physiques efficaces contre le Golem.

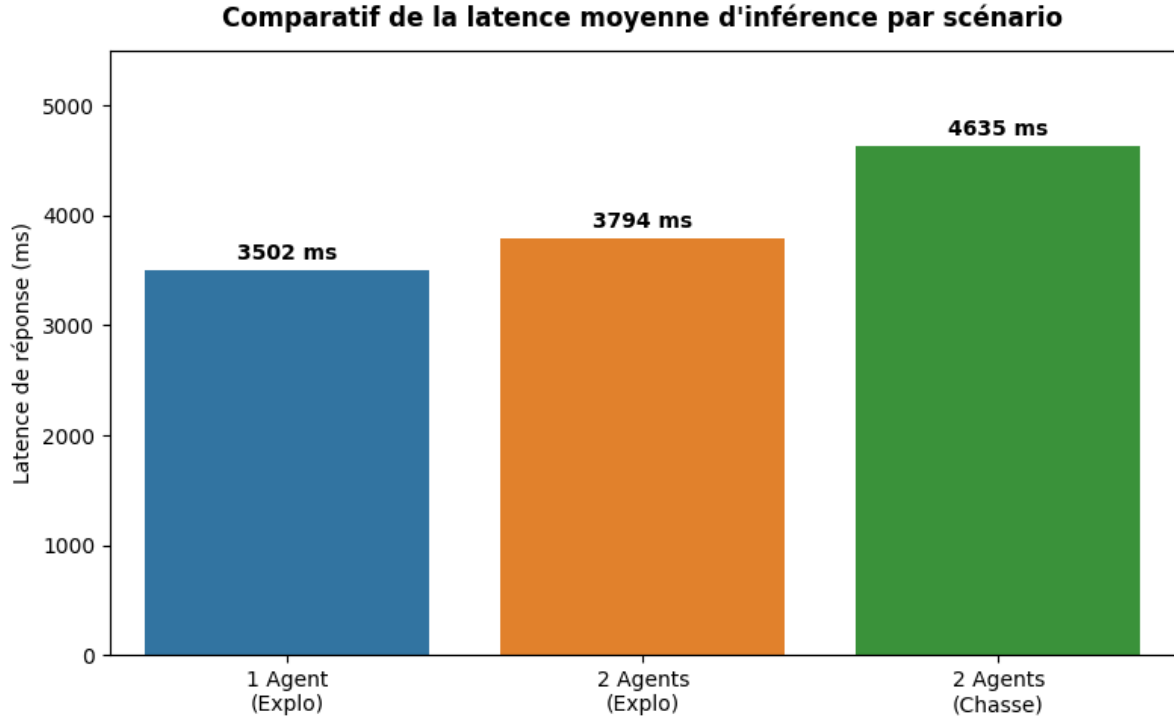


FIGURE 3 – Comparatif de la latence moyenne d'inférence par scénario (Hors Cold Start).

5.4 Le Crash-Test à l'Extrême : Simulation à 7 Agents simultanés

Afin de pousser notre architecture décentralisée dans ses derniers retranchements et d'identifier ses limites de rupture matérielles et logicielles, nous avons mené une simulation extrême en déployant simultanément **7 agents cognitifs** face au Golem. Les traces d'exécution obtenues mettent en évidence un effondrement du système :

1. **Hardware** : En configuration mono ou bi-agents, le mécanisme de KV-Cache d'Ollama garantissait une latence stable autour de 3.5 secondes. Lors du passage à 7 agents requêtant tous le même serveur local, la latence s'est envolée pour osciller entre **13 et 25 secondes par tour de décision**. Ce goulot d'étranglement n'est pas lié à un bug logiciel (les requêtes des agents restent parfaitement cohérentes), mais à l'incapacité matérielle à paralléliser 7 instances distinctes de modèles de 3 milliards de paramètres. Le serveur est contraint de sérialiser les inférences dans une file d'attente massive.
2. **Software** : Sur le plan du *Tool-Use*, l'architecture est restée saine : il n'y a eu aucun "spam d'outils", chaque agent n'appelant qu'une seule fonction de mouvement et de radio par itération. En revanche, l'effet cumulatif de la communication décentralisée a créé une surcharge fatale. Au bout de quelques tours, la boîte de réception (*Inbox*) de chaque agent s'est retrouvée saturée par les messages croisés des 6 autres alliés, tandis que la cartographie partagée dépassait les 80 arêtes. Subissant de plein fouet le phénomène de *Lost in the Middle* [7], la fenêtre de contexte de Llama 3.2 3B a saturé. Cette limite de contexte est une faille cognitive documentée où les LLMs perdent l'attention sur les informations situées au milieu d'un prompt long, tout en retenant parfaitement le début et la fin. Dans notre cas, l'agent se souvient

parfaitement de son rôle (au début du prompt) et de ses voisins immédiats (à la fin du prompt), mais perd l'historique topologique situé au centre de sa fenêtre de contexte. Ainsi, incapable de maintenir son attention jusqu'au bout des instructions, le modèle a commencé à halluciner des formats JSON invalides ou des identifiants de nœuds inexistants, déclenchant de manière répétée le message d'erreur : **Move failed. The AI might have provided a non-existent ID.**

Ce crash-test démontre empiriquement que l'architecture décentralisée pure trouve sa limite assez rapidement au niveau du nombre d'agents. Au-delà, la centralisation devient sûrement d'utilité absolue.

6 Discussion, Limites et Perspectives

L'intégration d'un Modèle de Langage Local au sein de la plateforme Dédale a permis de valider la faisabilité du concept d'*Agentic LLM* en environnement contraint, mais elle a également mis en exergue les limites actuelles de l'IA générative face aux algorithmes symboliques classiques.

6.1 Limites Cognitives et Logicielles

L'évaluation approfondie de notre architecture décentralisée a mis en exergue plusieurs verrous cognitifs inhérents à l'usage d'un *Small Language Model* (SLM) en environnement spatial dynamique.

1. **Fragilité du *Function Calling* et du formatage contraint** : La communication entre l'agent et le moteur Java repose sur un format JSON strict imposé par LangChain4j. Le modèle a pu fréquemment confondre la définition d'un schéma d'entrée textuel avec la valeur finale attendue (ex. : encapsulation incorrecte des arguments).
2. **L'effet *Lost in the Middle* et la saturation par le bruit radio** : Comme documenté par Liu et al. [7], les modèles de langage tendent à occulter les informations situées au centre de leur fenêtre de contexte. Dans notre configuration décentralisée, cette limite est exacerbée par la croissance du graphe global et l'accumulation des messages textuels dans l'*Inbox* (ex. : "OllamaBotX: Occupying node Y"). Noyé sous ce bruit, le SLM perd ses capacités de raisonnement. Il omet des instructions et dérive vers des hallucinations, proposant des identifiants de nœuds inexistants ou hors de portée.
3. **L'absence de modèle topologique** : Un modèle autoregressif prédit des probabilités linguistiques de tokens, mais ne possède aucune représentation géométrique ou matricielle de l'espace. Privé d'une structure de données interne dédiée (comme la pile d'un parcours DFS), l'agent est fondamentalement sans état (*stateless*).
4. **Limite drastique de scalabilité architecturale** : Le déploiement d'une instance de "cerveau" LLM par entité physique engendre un coût computationnel trop lourd en local. Lors du passage à une flotte moyenne (7 agents), le système subit un double effondrement : une asphyxie matérielle (file d'attente d'inférence sur le GPU poussant la latence à 25 secondes) et une asphyxie cognitive (contextes individuels saturés par la radio). L'approche décentralisée pure se heurte donc à un plafond de verre, limitant l'escouade à de très petits comités d'agents.

6.2 Perspectives d'Amélioration

Pour franchir un nouveau cap d'autonomie et d'efficacité, plusieurs évolutions architecturales sont envisageables :

1. **L'Hybridation Cognition/Navigation** : La perspective la plus prometteuse consisterait à retirer au LLM la charge de la navigation pure. Les déplacements physiques et l'exploration systématique (DFS, A*) seraient redélegués à des algorithmes Java déterministes. L'IA générative ne serait sollicitée que comme un "Général d'armée" pour des choix purement tactiques (ex : "Nous avons repéré la cible, calculez le point de convergence optimal").
2. **Implémentation de l'Architecture Centralisée (Ruche)** : Passer d'un modèle décentralisé à une architecture "Leader-Followers" permettrait de centraliser la prise de décision. Au lieu d'exécuter une inférence LLM par agent à chaque tour (soit n requêtes parallèles pour n agents), le Leader consoliderait les perceptions en un prompt unique. Cela diviserait la charge computationnelle proportionnellement au nombre d'agents déployés, tout en offrant au modèle une vision globale instantanée qui supprimerait les risques d'embouteillages ou de collisions entre alliés.
3. **Fine-Tuning du Modèle** : L'entraînement spécifique d'un petit modèle sur des jeux de données d'exploration spatiale et de réponses JSON strictes permettrait de se passer de l'ingénierie de prompt complexe, réduisant ainsi le nombre de tokens à traiter et accélérant davantage le temps de réponse.

7 Conclusion

Ce projet a démontré avec succès l'intégration d'un Modèle de Langage (LLM) exécuté localement au sein d'un Système Multi-Agents (SMA) déterministe, en utilisant l'API de JADE et de la plateforme Dédale. En construisant un pont logiciel via LangChain4j, nous avons réussi à traduire une perception physique sous forme de graphe en une abstraction sémantique compréhensible par l'intelligence artificielle.

Les expérimentations ont prouvé que l'approche de calcul en local est viable en temps réel pour la robotique simulée, avec des temps d'inférence stabilisés autour de 3.5 secondes, pour un modèle assez petit, grâce à l'exploitation du *Key-Value Cache*. De plus, nous avons identifié le problème d'absence de "mémoire à pile" (inhérente aux algorithmes d'exploration classiques) chez les modèles génératifs en forçant un mécanisme de *Chain-of-Thought*, alignant ainsi les probabilités linguistiques sur une stratégie spatiale plus cohérente.

Si l'IA symbolique conserve sa suprématie pour le calcul de chemins complexes, ce projet souligne le potentiel de l'IA générative pour la communication asynchrone entre agents, l'interprétation d'un environnement dynamique, et l'émergence de tactiques collaboratives dynamiques et autonomes. La création d'un système hybride, fusionnant la rigueur du code déterministe et l'adaptabilité sémantique des modèles de langage, constitue sans aucun doute une piste d'avenir de l'intelligence artificielle spatiale et multi-agents.

Références

- [1] "Plateforme Dédale, Environnement de simulation pour SMA." <https://dedale.gitlab.io/>.

- [2] A. Plaat, M. van Duijn, N. van Stein, M. Preuss, P. van der Putten, and K. J. Batenburg, “Agentic large language models, a survey,” *arXiv preprint arXiv :2503.23037*, 2025.
- [3] M. Morge, “Système multi-agents et agents autonomes génératifs : état de l’art des connaissances et des ressources,” in *33es Journées francophones sur les systèmes multi-agents (JFSMA)*, (Dijon (France)), 2025. HAL Id : hal-05146426.
- [4] AlexDuo, “LLM Unity AI Commander.” Hugging Face Blog. <https://huggingface.co/blog/AlexDuo/llm-unity-ai-commander>, 2024.
- [5] K.-T. Tran, D. Dao, M.-D. Nguyen, Q.-V. Pham, B. O’Sullivan, and H. D. Nguyen, “Multi-agent collaboration mechanisms : A survey of llms,” *arXiv preprint arXiv :2501.06322*, 2025.
- [6] “Hugging Face, Référentiel de modèles génératifs ouverts.” <https://huggingface.co/models>.
- [7] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle : How language models use long contexts,” *arXiv preprint arXiv :2307.03172*, 2023.